



UIT University, Karachi

PSEB INSPIRE Program
Digital IC Design & Verification

CAPSTONE PROJECT REPORT

RISC-V SoC with Network-on-Chip-Inspired Multi-Core Interconnect

Dual-Core RV32I SoC with AHB-Lite-Inspired Interconnect,
Shared Memory, Mailbox Communication, UART/GPIO Peripherals,
UVM Verification, ASIC Physical Design and FPGA Implementation



GROUP MEMBERS

Kashish Shaikh

Lareb Saba

Muskan Jarwar



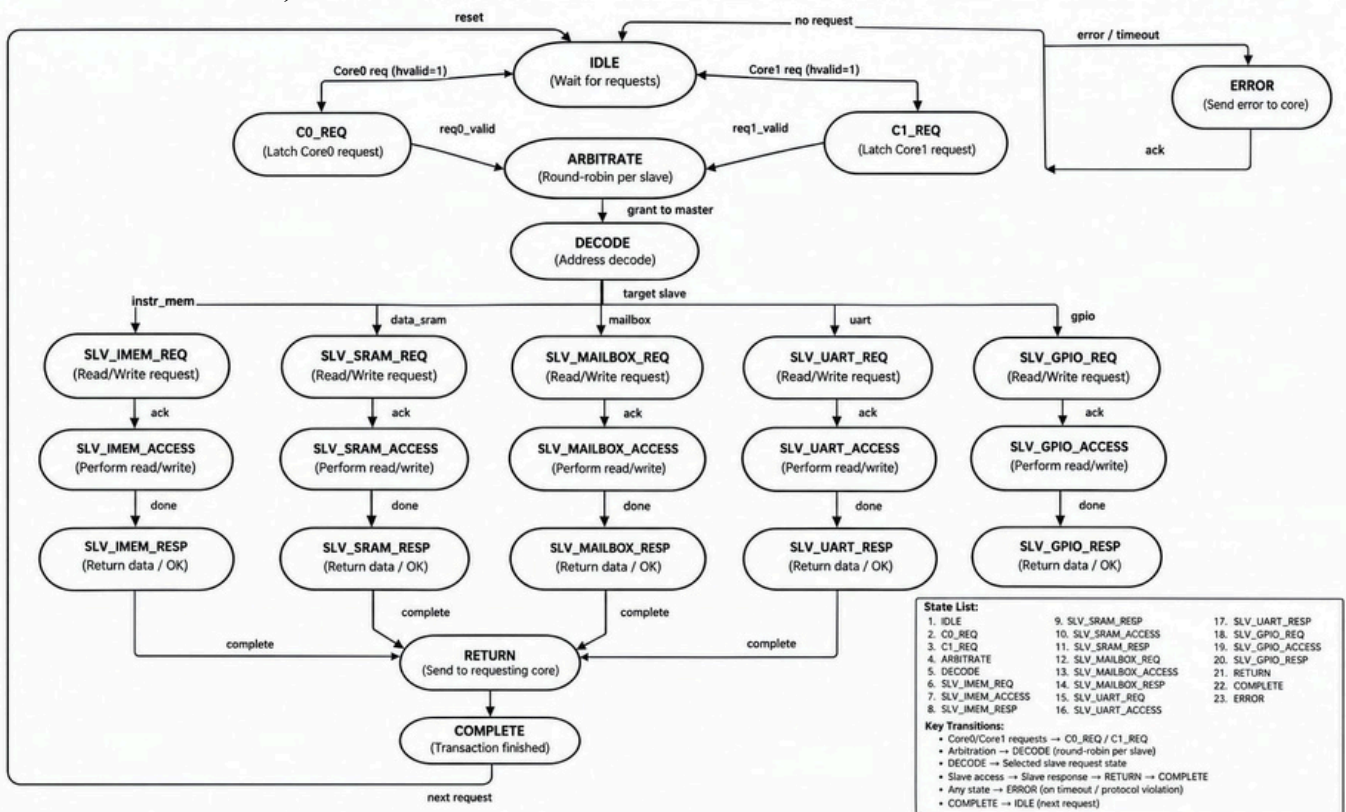
1. Project Objective

This project implements a dual-core RV32I RISC-V SoC with a structured AHB-Lite-inspired, NoC-style interconnect. The system supports shared-memory communication, inter-core mailbox communication, memory-mapped UART/GPIO peripherals, round-robin arbitration, UVM verification, ASIC physical implementation, and FPGA demonstration.

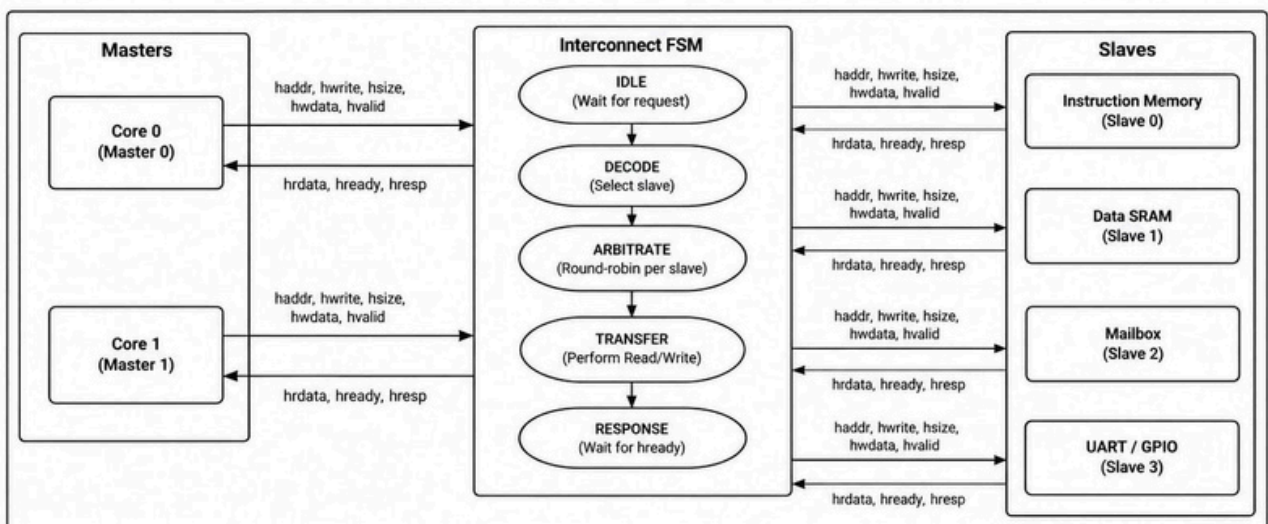
Key Features

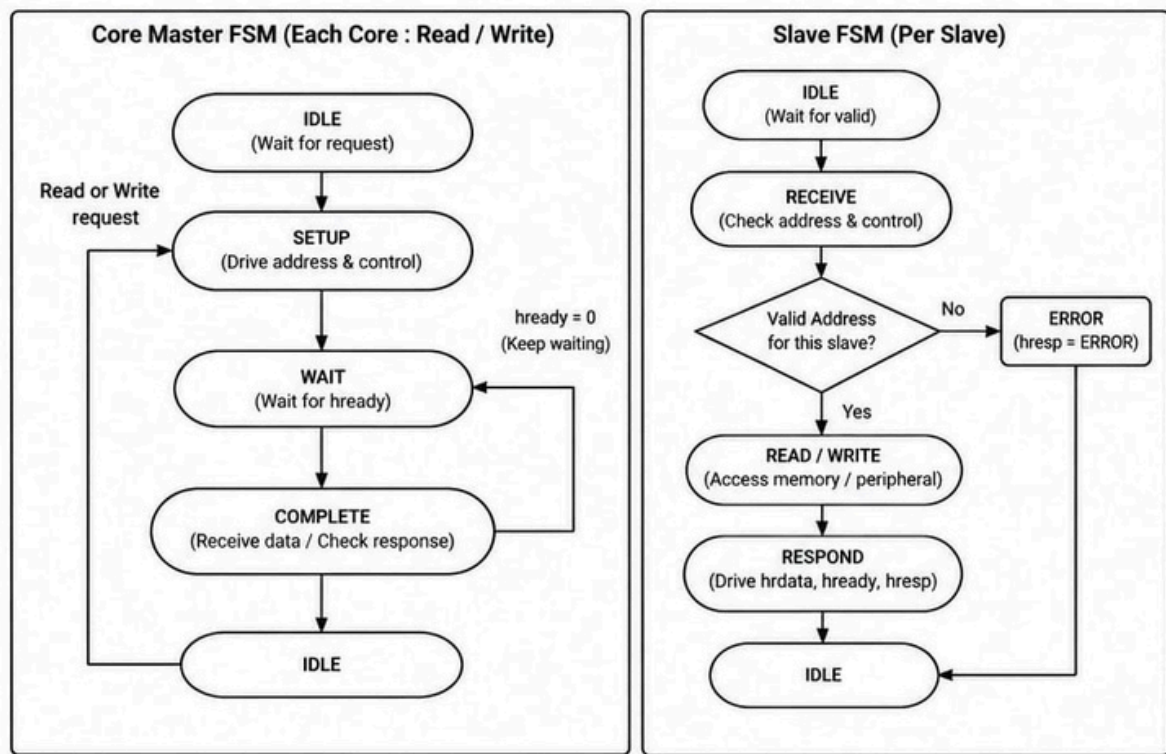
- Two synthesizable single-cycle RV32I cores
- Private instruction memories and shared 8 KB data SRAM
- Address-based routing with four memory/peripheral targets
- Per-slave round-robin arbitration
- Inter-core mailbox communication
- Memory-mapped UART and GPIO/LED
- Directed self-checking verification
- UVM-1.1 verification with scoreboard, coverage and SVA
- Sky130A RTL-to-GDSII implementation
- Arty A7-100T FPGA demonstration

FSM (Finite State Machine)



Top-Level FSM – AHB-Lite Interconnect (2 Masters : Core 0 & Core 1)

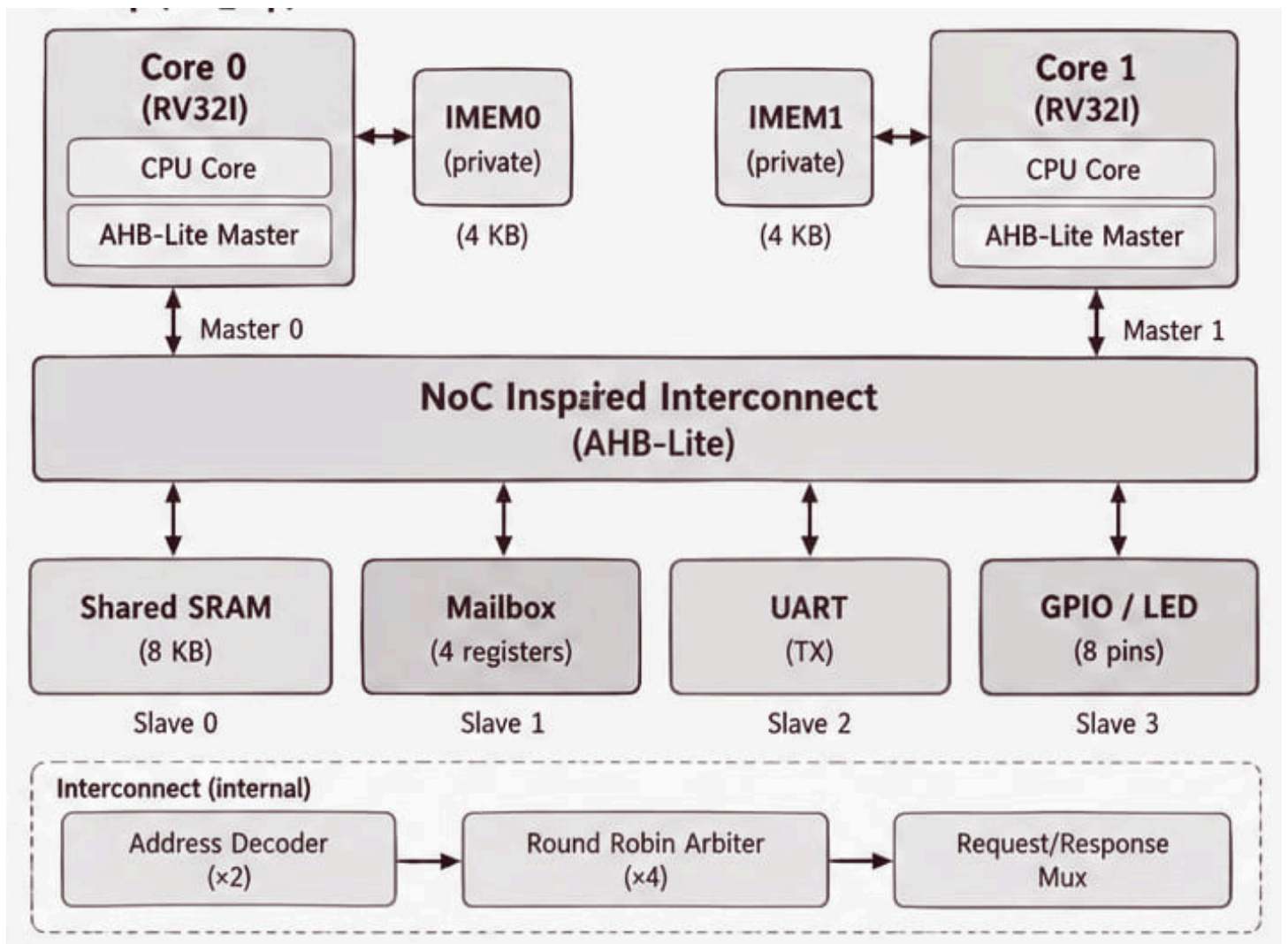




2. System Architecture

The SoC consists of two independent RV32I cores. Each core has its own instruction memory and communicates with shared resources through the interconnect.

The interconnect performs address decoding, per-slave arbitration and response multiplexing. Requests targeting different slaves can execute in parallel, while simultaneous requests to the same slave are resolved using round-robin arbitration.



Main Components:

2.1 Top-Level Block Diagram

Layer	Component	Role
Compute	rv32i_core (×2)	Single-cycle RV32I core; fetches from private IMEM, issues AHB-Lite data transfers
Memory (private)	imem (×2)	Per-core instruction ROM, combinational read, \$readmemh-loaded test programs
Interconnect	addr_decoder (×2, one per master)	Classifies each master's address into a target slave or 'unmapped'
Interconnect	arbiter (×4, one per slave)	Round-robin grant between master0/master1 when both target the same slave
2.1 Top-Level Block Diagram	noc_interconnect	Top-level router: combines decoders + arbiters, muxes requests/responses
Shared Memory	shared_dmem	8 KB shared data SRAM, byte/half/word AHB-Lite slave
Inter-core Comms	ic_mailbox	4 registers (2 data + 2 flag) for software-driven core-to-core messaging
Peripheral	uart_peripheral	Hardware UART transmitter (start/8-data/stop FSM), TXDATA + STATUS registers
Peripheral	gpio_peripheral	Memory-mapped LED output register, 8 physical pins
Top	soc_top	Instantiates both cores, both IMEMs, interconnect, and all four slaves

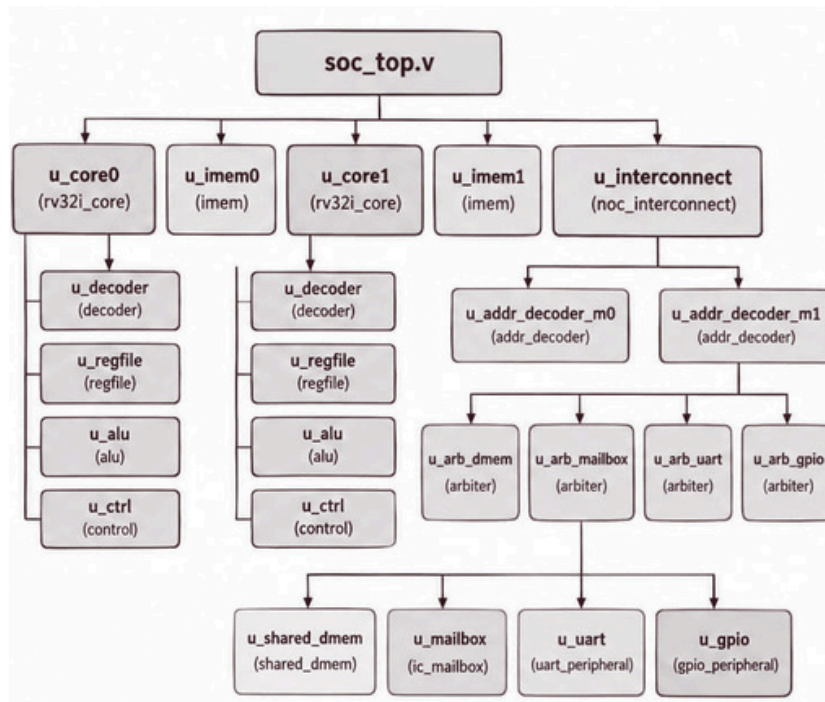
2.2 System Address Map

Region	Base Address	Size	Notes
Core 0 / Core 1 Instruction Memory	0x0000_0000	4 KB each	Private per-core, not on the shared interconnect
Shared Data SRAM	0x1000_0000	8 KB	Byte / half / word accessible, arbitrated between both cores
Inter-Core Mailbox	0x2000_0000	256 B	MBOX_C0_TO_C1_DATA/FLAG
UART Peripheral	0x3000_0000	256 B	UART_TXDATA (0x00), UART_STATUS (0x04)
GPIO / LED Peripheral	0x3000_0100	256 B	GPIO_LED register, 8 output pins
Unmapped	all other addresses	—	Interconnect returns HRESP_ERROR with zero wait states

3. RTL Design and Code Hierarchy

All RTL is written in plain synthesizable Verilog (IEEE 1364-2005), with a single shared header (rv32i_soc_defines.v) providing the address map, response/size codes, and RV32I opcode constants to every module via `include, since plain Verilog has no package construct.

3.1 Module Hierarchy



3.2 Verilog Source File Summary

File	Purpose
rv32i_soc_defines.v	Global address map, response/size codes, RV32I opcode macros
regfile.v	32×32-bit RV32I register file, x0 hardwired to zero
alu.v	10-operation combinational ALU (ADD/SUB/logic/shifts/SLT/SLTU)
decoder.v	RV32I instruction decode + control-signal + immediate generation
rv32i_core.v	Single-cycle core top-level: PC, branch/jump logic, AHB-Lite master, wait
imem.v	Private per-core instruction ROM (parameterised depth, \$readmemh init)
shared_dmem.v	8 KB shared data SRAM, AHB-Lite slave, byte/half/word write support
ic_mailbox.v	4-register inter-core mailbox (2 data + 2 flag registers)
uart_peripheral.v	UART TX FSM (start/8-data/stop), TXDATA/STATUS registers
gpio_peripheral.v	Memory-mapped LED output register
addr_decoder.v	Per-master combinational address→slave-select decoder
arbiter.v	Per-slave 2-master round-robin arbiter
noc_interconnect.v	Top-level router: decode, per-slave arbitration, request/response muxing
soc_top.v	Top-level integration of both cores, IMEMs, interconnect, and 4 slaves

4. Bus Protocol

Communication between the cores, the shared-memory subsystem, and the memory-mapped peripherals uses an AHB-Lite-inspired protocol, implemented as a flattened (non-struct) signal set per master/slave port: haddr, hwrite, hsize, hwdata, hvalid (request), and hrdata, hready, hresp (response). The protocol is simplified for single-outstanding, non-pipelined masters, matching the single-cycle core microarchitecture.

Signal	Direction	Description
haddr[31:0]	$M \rightarrow S$	Byte address of the current transfer
hwrite	$M \rightarrow S$	1 = write, 0 = read
hsize[1:0]	$M \rightarrow S$	00 = byte, 01 = halfword, 10 = word
hwdata[31:0]	$M \rightarrow S$	Write data (byte/halfword lane-replicated by the master)
hvalid	$M \rightarrow S$	Transfer request asserted; held until hready
hrdata[31:0]	$S \rightarrow M$	Read data, valid when hready is asserted
hready	$S \rightarrow M$	1 = transfer complete this cycle
hresp	$S \rightarrow M$	0 = HRESP_OKAY, 1 = HRESP_ERROR (unmapped access)

5. Basic Self-Checking Verification

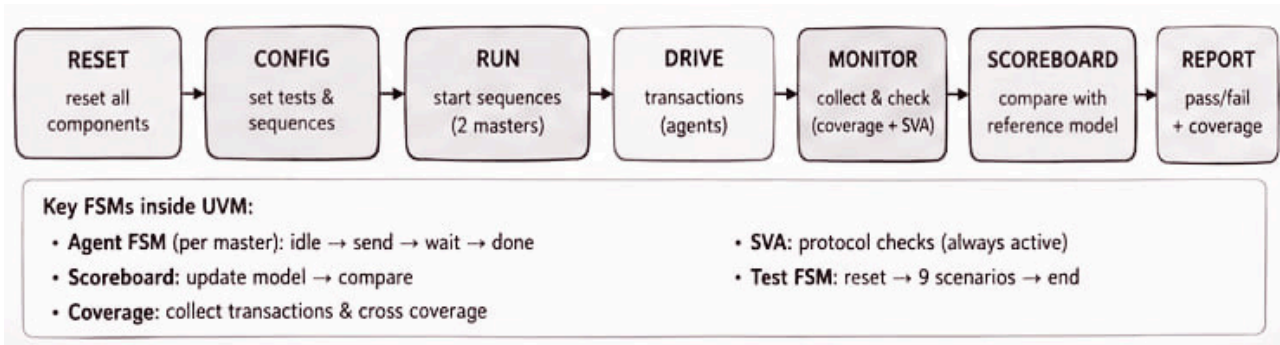
Prior to UVM, a directed, self-checking Verilog/SystemVerilog testbench (tb_noc_interconnect.sv) verified the interconnect and all four slaves by driving both AHB-Lite masters directly (bypassing the cores for full timing control), with a pass/fail scoreboard task built into the testbench itself.

5.1 Directed Test Cases and Results

#	Test Case	What is Verified	Result
1	Reset	All slaves idle, mailbox/LED clear after reset	PASS
2	Simultaneous Access / Arbitration	Forced same-cycle contention on shared SRAM; round-robin grant, 1-cycle stall for loser	PASS
3	Mailbox Communication	Core0→Core1 data+flag write, Core1 read, flag ack-clear	PASS
4	UART Transmission	Byte written to TXDATA; serial start/8-data/stop bits independently decoded and verified	PASS
5	GPIO / LED	Register write + physical pin readback	PASS
6	Invalid / Unmapped Address	Out-of-range access returns HRESP_ERROR with 0 wait states	PASS
7	Back-to-Back Access	Consecutive same-master transactions, no data corruption	PASS
8	Parallel Routing (NoC Property)	Two masters target different slaves in the same cycle — both complete with 0 stall	PASS

6. UVM Verification Environment

A complete UVM-1.1 environment was built to verify concurrent transactions from both cores, interconnect routing, arbitration fairness, shared-memory and mailbox communication, back-to-back accesses, reset behaviour, and error handling — using class-based agents, sequences, a reference-model scoreboard, functional coverage, and SystemVerilog Assertions (SVA).



A single top-level environment instantiates two independent, identical master agents (one per core-equivalent AHB-Lite interface) plus a shared scoreboard and coverage collector — rather than one isolated UVM environment per RTL block. This is deliberate: arbitration, contention, and interconnect routing are inter-master phenomena, and can only be exercised and observed with both masters live in the same environment.

`ahb_transaction` — sequence-item class (addr, write, wdata, size, rdata, resp, wait_cycles, master_id).

`ahb_sequencer` / `ahb_driver` / `ahb_monitor` — one set per master, driving/observing the AHB-Lite virtual interface.

`ahb_agent` (× 2) — bundles sequencer + driver + monitor per master; `UVM_ACTIVE`.

`ahb_scoreboard` — reference-model based: mirrors shared SRAM / mailbox / GPIO writes internally and compares every subsequent read against the model.

`ahb_coverage` — functional covergroup: master, target slave, direction, response, arbitration wait-cycles, and three cross-coverages.

Protocol assertions (SVA) — embedded directly in the `ahb_if` interface, automatically applied to both master interface instances.

`ahb_base_test` — orchestrates 9 scenario sequences against the two agents' sequencers.

Scenario	Purpose
Reset	Deterministic idle state on all slaves post-reset
Simultaneous Access / Arbitration (+ round-robin flip)	Forced same-cycle contention on shared SRAM twice in a row, proving fairness alternates (Master0 stalls first, then Master1)
Mailbox Inter-Core Communication	Full handshake: write data+flag, poll/read, ack-clear
UART Peripheral	Byte transmission, busy-flag polling, second master status read
GPIO / LED	Cross-master write/readback
Invalid / Unmapped Address	<code>HRESP_ERROR</code> from both masters
Back-to-Back Access	Consecutive same-master transactions
Parallel Routing to Different Slaves	Genuine NoC-style zero-stall parallel access, proven for both masters

6.3 Scoreboard, Coverage, and Assertion Results

Metric	Result
Scoreboard checks (auto-generated from reference model)	28 / 28 PASS, 0 FAIL
Functional coverage (master × slave × direction × response × wait-cycles, incl. 3 cross-coverages)	100.00%
SVA protocol assertions (reset-clears-valid, no unknown address, stable-during-wait, no-indefinite-hang)	0 violations across full regression
UVM report summary	UVM_ERROR : 0, UVM_FATAL : 0

7. Physical Design Implementation

The complete top-level design (soc_top) was carried through synthesis and a full RTL-to-GDSII physical-design flow using OpenLane/LibreLane on the open-source SkyWater Sky130A PDK (sky130_fd_sc_hd standard-cell library), covering floorplanning, power-grid generation, placement, clock-tree synthesis, global/detailed routing, parasitic extraction, and post-route static timing sign-off.

Parameter	Value
PDK / Standard-Cell Library	sky130A / sky130_fd_sc_hd
Target Clock Period	20.0 ns (50 MHz)
Placement Target Density	40%
Die Area	75.75 μm × 86.47 μm (6550.1 μm²)
Core Area	4028.86 μm² (35.7 % utilisation)
Flow Stages Executed	Synthesis → Floorplan → PDN → Placement → CTS → Global/Detailed Routing → Parasitic Extraction (RCX) → Post-Route

Sign-Off Results

Check	Result
Setup timing (WNS / TNS)	0.000 ns / 0.000 ns (worst-case setup slack = 13.73 ns) — MET
Hold timing (WNS / TNS)	0.000 ns / 0.000 ns (worst-case hold slack = 0.107 ns) — MET
Worst clock skew (setup / hold)	0.203 ns / -0.203 ns
Routing DRC violations	0
Antenna violations	0
Total instance count (incl. fill/tap/decap)	892
Total estimated power	≈ 0.114 mW (post-route, default activity)

8. FPGA Implementation

The dual-core SoC was implemented on the Digilent Arty A7-100T board (Xilinx xc7a100t) using the open-source F4PGA/Symbiflow flow (Yosys synthesis → VPR placement/routing → FASM → bitstream), demonstrating both cores executing independent programs and communicating via shared memory and the mailbox, with LEDs and UART used to show live core/interconnect activity.

8.1 Board I/O Mapping

Signal	FPGA Pin	Function
CLK100MHZ	E3	100 MHz system clock
btn_reset	D9	Active-high reset button
uart_tx	A9	USB-UART bridge TX (byte transmission demo)
led[3:0]	H5 / J5 / T9 / T10	Software-controlled GPIO output
led[4]	E1	Core0 → Core1 mailbox flag (live indicator)
led[5]	G4	Core1 → Core0 mailbox flag (live indicator)

9. Final Deliverables Checklist

Required Deliverable	Status	Location / Evidence
Synthesizable RTL	Complete	14 Verilog source files, soc_top.v top level
Dual RV32I core integration	Complete	u_core0 / u_core1 in soc_top.v
Instruction / shared SRAM integration	Complete	imem.v (×2, private), shared_dmem.v (shared, arbitrated)
Structured multi-core interconnect	Complete	noc_interconnect.v + addr_decoder.v + arbiter.v
Bus / peripheral interface	Complete	AHB-Lite–inspired protocol; ic_mailbox.v, uart_peripheral.v, gpio_peripheral.v
FPGA demonstration	Complete	Arty A7-100T bitstream, F4PGA flow reports
Basic self-checking simulation	Complete	tb_noc_interconnect.sv — 26/26 PASS
UVM verification environment	Complete	Agents, sequences, scoreboard (28/28 PASS), 100% functional coverage, SVA assertions
Timing constraints	Complete	ASIC SDC (20 ns) and FPGA XDC/SDC (10 ns) constraint sets
Synthesis and physical-design reports	Complete	OpenLane/LibreLane per-stage reports, final metrics.json
Coverage reports	Complete	UVM functional coverage (100%) + code coverage flow configured
Final layout / GDS	Complete	GDSII generated via magic-streamout stage
Architecture document	Complete	This document

10. Results

1. System Verilog Self-Checking Verification :

```
TEST 2 : SIMULTANEOUS ACCESS / ARBITRATION TO SHARED SRAM
[PASS] master0 wait_cycles = 1
[PASS] master1 wait_cycles = 0
[PASS] master1 wins arbitration first (0 wait cycles) : got=0x00000000 expected=0x00000000
[PASS] master0 stalls exactly 1 cycle (loses first round) : got=0x00000001 expected=0x00000001
[PASS] shared_dmem[0] readback (written by master0) : got=0xaaaa0000 expected=0xaaaa0000
[PASS] shared_dmem[1] readback (written by master1) : got=0xbbbb1111 expected=0xbbbb1111

TEST 3 : MAILBOX INTER-CORE COMMUNICATION
[PASS] mbox_c0_to_c1_flag asserted after core0 write : got=0x00000001 expected=0x00000001
[PASS] core1 reads correct mailbox data : got=0xcafebabe expected=0xcafebabe
[PASS] core0 reads flag = 1 before ack : got=0x00000001 expected=0x00000001
[PASS] mbox_c0_to_c1_flag cleared after core1 ack : got=0x00000000 expected=0x00000000

TEST 4 : UART PERIPHERAL BYTE TRANSMISSION
[PASS] UART transmitted byte matches 0x41 ('A') : got=0x00000041 expected=0x00000041
[PASS] UART stop bit / frame valid : got=0x00000001 expected=0x00000001
[PASS] UART busy flag cleared after transmission : got=0x00000000 expected=0x00000000

TEST 5 : GPIO / LED PERIPHERAL
[PASS] GPIO register readback : got=0x000000a5 expected=0x000000a5
[PASS] physical led pins reflect written value : got=0x000000a5 expected=0x000000a5

TEST 6 : INVALID / UNMAPPED ADDRESS ACCESS
[PASS] unmapped access completes with 0 wait cycles : got=0x00000000 expected=0x00000000
[PASS] unmapped access returns HRESP_ERROR : got=0x00000001 expected=0x00000001
[PASS] unmapped write also returns HRESP_ERROR : got=0x00000001 expected=0x00000001

TEST 7 : BACK-TO-BACK ACCESSES (SAME MASTER)
[PASS] back-to-back write 1 readback : got=0x11112222 expected=0x11112222
[PASS] back-to-back write 2 readback : got=0x33334444 expected=0x33334444

TEST 8 : PARALLEL ROUTING TO DIFFERENT SLAVES (NOC PROPERTY)
[PASS] master0 -> DMEM sees no stall despite same-cycle master1 request : got=0x00000000 expected=0x00000000
[PASS] master1 -> MAILBOX sees no stall despite same-cycle master0 request : got=0x00000000 expected=0x00000000
[PASS] parallel DMEM write landed correctly : got=0x55556666 expected=0x55556666
[PASS] parallel MAILBOX write landed correctly : got=0x77778888 expected=0x77778888

TEST SUMMARY : PASS=26 FAIL=0

** Note: $finish : /home/cl4/Desktop/riscv_soc_project/tb_noc_interconnect.sv(394)
Time: 91690 ns Iteration: 1 Instance: /tb_noc_interconnect
```

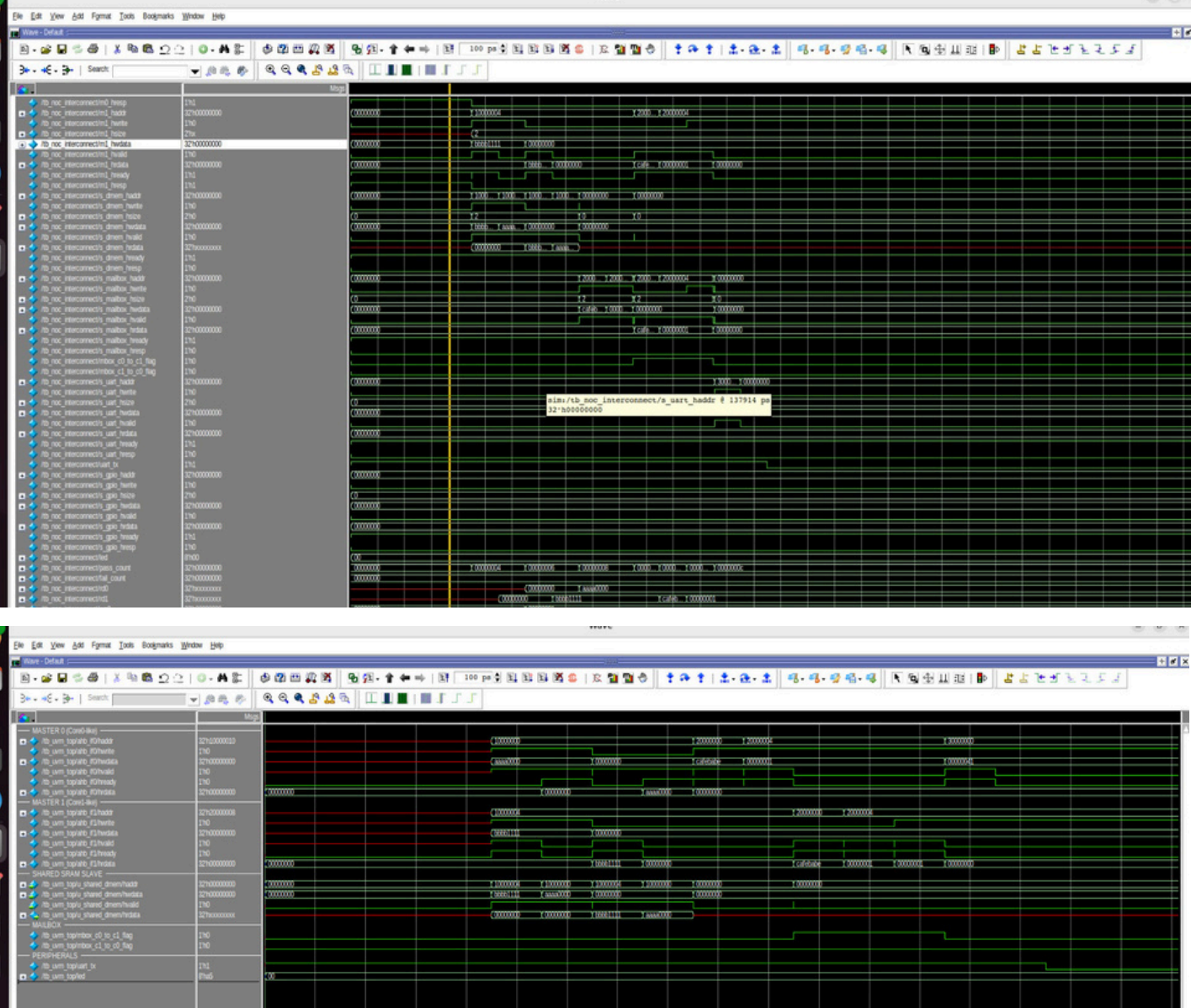
2. UVM Verification Results:

The screenshot displays a UVM verification environment. The top panel shows the project structure with files like `ahb_driver.sv`, `ahb_monitor.sv`, `ahb_base_test.sv`, `tb_uvm_top.sv`, `uvm_root.svh`, `ahb_agent.sv`, and `ahb_env.sv`. The main window shows a transcript of the verification process, including the following summary:

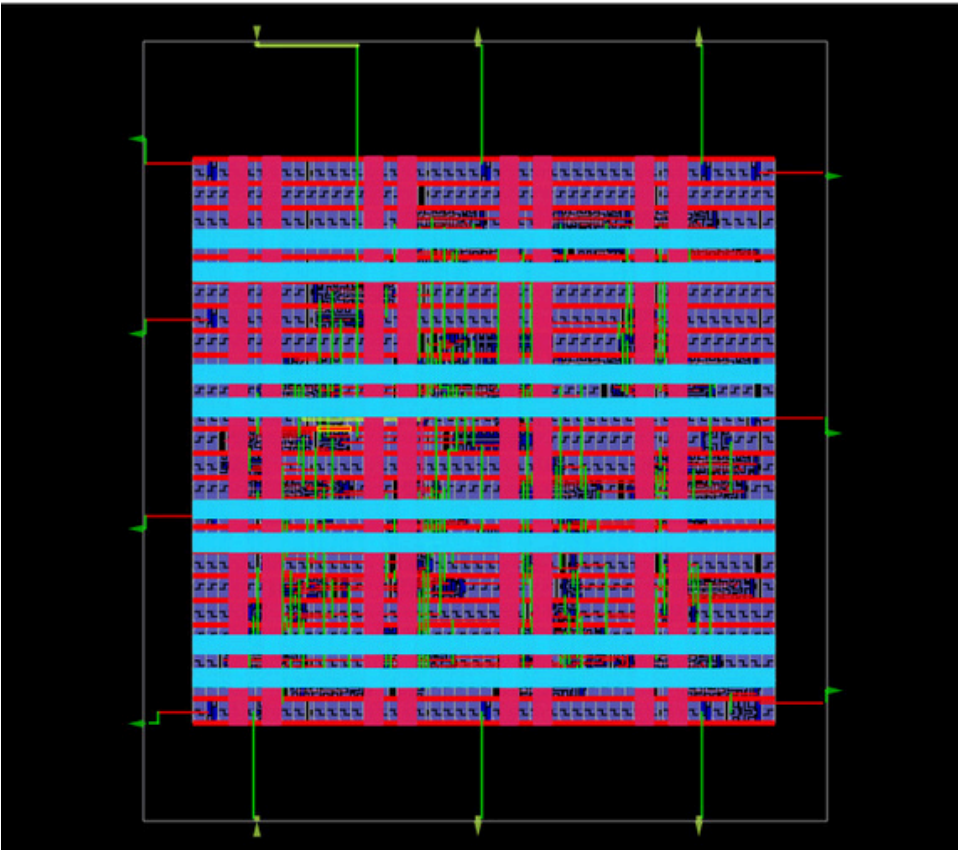
```
--- UVM Report Summary ---
** Report counts by severity
UVM_INFO : 46
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0
** Report counts by id
[COVERAGE] 1
[Questa UVM] 2
[RNTST] 1
[SCOREBOARD] 29
[TEST] 12
[TEST_DONE] 1
** Note: $finish : /home/cl4/questasim/linux_x86_64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
Time: 88530 ns Iteration: 54 Instance: /tb_uvm_top
1
Break in Task uvm_pkg/uvm_root::run_test at /home/cl4/questasim/linux_x86_64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```

The bottom status bar indicates the project is `riscv_soc_project`, the time is `Now: 88,530 ns`, and the delta is `Delta: 54`. The simulation is running on `sim:/tb_uvm_top@INITIAL194`.

3.Simulation Results:



4.GDS Results:



```
[INFO GRT-0001] Minimum degree: 2
[INFO GRT-0002] Maximum degree: 14

[INFO GRT-0053] Routing resources analysis:
Routing      Original      Derated      Resource
Layer  Direction  Resources    Resources    Reduction (%)
-----
li1     Vertical    0            0            0.00%
met1    Horizontal 1711         1142         33.26%
met2    Vertical   1415         951          32.79%
met3    Horizontal 1047         701          33.05%
met4    Vertical   652          390          40.18%
met5    Horizontal 144          99           31.25%
-----

[INFO GRT-0197] Via related to pin nodes: 624
[INFO GRT-0198] Via related Steiner nodes: 13
[INFO GRT-0199] Via filling finished.
[INFO GRT-0111] Final number of vias: 766
[INFO GRT-0112] Final usage 3D: 2601

[INFO GRT-0096] Final congestion report:
Layer      Resource      Demand      Usage (%)      Max H / Max V / Total Overflow
-----
li1         0             0            0.00%          0 / 0 / 0
met1       1142          146         12.78%          0 / 0 / 0
met2       951           146         15.35%          0 / 0 / 0
met3       701            7            1.00%          0 / 0 / 0
met4       390            4            1.03%          0 / 0 / 0
met5        99            0            0.00%          0 / 0 / 0
-----
Total      3283          303          9.23%          0 / 0 / 0
```

```
00:25:19] VERBOSE Logging subprocess to 'runs/RUN_2026-09-07_05-45-49/385-openroad-stapostpr/mln_ff_n48C_iv95/filter_unannotated.log'... step.py:133
00:25:19] VERBOSE Logging subprocess to 'runs/RUN_2026-09-07_05-45-49/385-openroad-stapostpr/mln_tt_025C_iv80/filter_unannotated.log'... step.py:133
00:25:19] VERBOSE Logging subprocess to 'runs/RUN_2026-09-07_05-45-49/385-openroad-stapostpr/max_ss_100C_iv60/filter_unannotated.log'... step.py:133

Generate IR Drop Report
-----
Corner/Group  Hold Worst Slack  Reg to Reg Paths  Hold THS  Hold Vio Count  of which reg to reg  Setup Worst Slack  Reg to Reg Paths  Setup THS  Setup Vio Count  of which reg to reg  Max Cap Violations  Max Slew Violations
-----
Overall       0.1065            0.1065            0.0000     0              0              0              13.7264            13.7264            0.0000     0              0              0              0
non_tt_025C_iv80 0.2824            0.2824            0.0000     0              0              0              16.8838            17.1007            0.0000     0              0              0              0
non_ss_100C_iv60 0.7871            0.7871            0.0000     0              0              0              13.7435            13.7435            0.0000     0              0              0              0
non_ff_n48C_iv95 0.1088            0.1088            0.0000     0              0              0              17.1706            18.1925            0.0000     0              0              0              0
min_tt_025C_iv80 0.2804            0.2804            0.0000     0              0              0              16.8893            17.1123            0.0000     0              0              0              0
min_ss_100C_iv60 0.7831            0.7831            0.0000     0              0              0              13.7596            13.7596            0.0000     0              0              0              0
min_ff_n48C_iv95 0.1065            0.1065            0.0000     0              0              0              17.1739            18.2029            0.0000     0              0              0              0
max_tt_025C_iv80 0.2842            0.2842            0.0000     0              0              0              16.7991            17.0850            0.0000     0              0              0              0
max_ss_100C_iv60 0.7903            0.7903            0.0000     0              0              0              13.7264            13.7264            0.0000     0              0              0              0
max_ff_n48C_iv95 0.1094            0.1094            0.0000     0              0              0              17.1667            18.1817            0.0000     0              0              0              0

00:25:20] VERBOSE Running 'OpenROAD_IRDropReport' at 'runs/RUN_2026-09-07_05-45-49/386-openroad-irdropreport'... step.py:113
00:25:20] WARNING 'VSRC_LOC_FILES' was not given a value, which may make the results of IR drop analysis inaccurate. If you are not integrating a top-level chip for openroad.py:224
```

5.FPGA Results:

